



دانشگاه تربیت مدرس
دانشکده مهندسی برق و کامپیوتر

گزارش تمرین اول

درس یادگیری عمیق (Deep Learning)

نگارنده:

محمد رضا قادری عمودیزج
شماره دانشجویی: ۴۰۲۶۱۰۵۱۰۰۴

مقطع:

کارشناسی ارشد مهندسی کامپیوتر

نیم سال تحصیلی ۱۴۰۴-۱۴۰۵

۱ مقدمه و منطق جمع‌آوری خودکار داده‌ها

با توجه به خواسته تمرین مبنی بر ایجاد یک مجموعه داده با حداقل ۱۰۰ رکورد از اطلاعات خودروهای فروشی (شامل بیش از ۱۰ ویژگی مختلف برای هر ماشین)، ساده‌ترین راه در نگاه اول کپی کردن دستی اطلاعات از سایت‌های نیازمندی مانند دیوار بود. اما انجام دستی این کار چند مشکل اساسی داشت:

- **احتمال بالای خطای انسانی:** وارد کردن دستی حدود ۱۵۰۰ فیلد اطلاعاتی مختلف، احتمال اشتباه تایپی (جابه‌جا وارد کردن قیمت، کارکرد یا سال تولید) را به شدت افزایش می‌دهد که این موضوع در مرحله آموزش مدل باعث ایجاد نویز و افت دقت می‌شود.

- **زمان‌بر بودن:** استخراج دستی اطلاعات و خواندن دقیق متن توضیحات برای پیدا کردن ویژگی‌هایی مثل «ایر‌بگ» یا «وضعیت لاستیک» نیازمند صرف ساعت‌ها زمان است.

- **عدم مقیاس‌پذیری:** در صورتی که در طول انجام پروژه متوجه می‌شدیم به داده‌های بیشتری نیاز داریم (مثلاً ۲۰۰ یا ۳۰۰ رکورد برای بهبود مدل)، باید مجدداً فرآیند خسته‌کننده دستی را تکرار می‌کردیم.

به همین دلایل، تصمیم بر آن شد تا یک اسکریپت وب‌اسکرپینگ به زبان پایتون نوشته شود تا داده‌ها به صورت کاملاً خودکار، ساختاریافته و با بالاترین دقت ممکن استخراج شوند. در نهایت تعداد ۱۵۰ رکورد از خودروی «تارا» از پلتفرم دیوار جمع‌آوری شد.

۲ نحوه کارکرد اسکریپت استخراج داده (Scraping)

سایت‌های آگهی مانند دیوار برای جلوگیری از استخراج داده توسط ربات‌ها، از لایه‌های امنیتی مختلفی (مانند تغییر مداوم ساختار HTML و سیستم‌های شناسایی ربات) استفاده می‌کنند. برای دور زدن این محدودیت‌ها و استخراج تمیز اطلاعات، رویکرد زیر در کد پیاده‌سازی شد:

۱. **شبیه‌سازی مرورگر واقعی:** به جای استفاده از درخواست‌های ساده، از کتابخانه‌هایی استفاده شد که اثر انگشت (Fingerprint) مرورگر کروم را شبیه‌سازی می‌کنند تا سیستم امنیتی سایت متوجه خودکار بودن فرآیند نشود.

۲. **استفاده از API داخلی به جای HTML:** به جای جستجو در تگ‌های تو در توی وب‌سایت که مدام در حال تغییر هستند، اسکریپت مستقیماً با API بک‌اند سایت ارتباط برقرار می‌کند. با این کار، اطلاعات هر ماشین به صورت یک بسته داده‌ی خام و تمیز (JSON) دریافت می‌شود.

۳. **پردازش متن توضیحات فروشنده:** ویژگی‌هایی مانند «وضعیت لاستیک‌ها» یا «وجود ایربگ» معمولاً در جدول مشخصات سایت دیوار وجود ندارند و فروشنده آن‌ها را در متن توضیحات می‌نویسد. در اسکریپت توابعی نوشته شد که متن توضیحات را اسکن کرده و این ویژگی‌ها را با استفاده از کلمات کلیدی استخراج کند.

۳ ساختار داده‌های ذخیره شده (Dataset)

پس از پردازش اطلاعات دریافت شده، داده‌ها به منظور استفاده در مدل‌های یادگیری ماشین، در یک فایل CSV با نام `divar_tara_ml_dataset.csv` ذخیره شدند. فرمت CSV بهترین استاندارد برای این کار است زیرا به سادگی توسط کتابخانه‌ی Pandas در پایتون بارگذاری می‌شود.

هر سطر از این فایل نمایانگر یک خودروی مجزا است و ستون‌های آن دقیقاً مطابق با نیازمندی‌های تمرین نگاشت شده‌اند:

- **مشخصات عددی:** قیمت، کارکرد (کیلومتر)، سال تولید و سن خودرو.
 - **وضعیت سلامت:** بدنه، وضعیت موتور، وضعیت شاسی‌ها و گیربکس.
 - **مشخصات جانبی:** رنگ، نوع سوخت، مهلت بیمه، وضعیت لاستیک و ایربگ.
- در این مجموعه داده تلاش شده است تا داده‌های ناقص حذف شوند و متن‌های فارسی (مانند اعداد فارسی) برای جلوگیری از خطاهای پردازشی در آینده، پاک‌سازی و به اعداد استاندارد انگلیسی تبدیل شوند.

۴ ایجاد مجموعه داده و پیش‌پردازش

در این تمرین، هدف بررسی قیمت خودروی «تارا» و پیش‌بینی آن بر اساس ویژگی‌های مختلف است. برای جمع‌آوری داده‌ها، به جای استخراج دستی که فرآیندی بسیار زمان‌بر و مستعد خطای انسانی (مانند اشتباه در وارد کردن اعداد طولانی قیمت) است، از یک اسکریپت خودکار برای استخراج داده‌ها (Web Scraper) استفاده شد.

این روش به ما اجازه داد تا در مدت زمانی کوتاه، اطلاعات دقیق ۱۵۰ خودرو را مستقیماً از پلتفرم دیوار دریافت کنیم. داده‌های استخراج شده برای دسترسی آسان در مراحل بعدی و قابلیت بارگذاری در کتابخانه‌های پردازشی، در قالب یک فایل CSV ذخیره شدند. این فرمت به دلیل ساختار جدولی، بهترین گزینه برای نگهداری ویژگی‌های عددی و متنی به صورت همزمان است.

۵ مهندسی ویژگی‌ها و رگرسیون خطی

- پس از آماده‌سازی فایل داده‌ها، مراحل زیر جهت بهبود کیفیت یادگیری انجام شد:
- **پاک‌سازی داده‌ها:** ستون‌هایی که تأثیری در قیمت نداشتند (مانند لینک آگهی و توکن) حذف شدند. همچنین با توجه به وجود ستون «سن خودرو»، ستون «سال تولید» برای جلوگیری از تکرار داده‌های مشابه حذف گردید.
 - **حذف نویز:** در داده‌های وب، همواره مقادیر پرت وجود دارند. با اعمال فرمول $Q1 - 1.5 \times IQR$ و $Q3 + 1.5 \times IQR$ ، اسکریپت موفق شد ۹ رکورد نویز (شامل قیمت‌های بسیار پایین یا غیرمنطقی) را شناسایی و از چرخه آموزش حذف کند.

۱.۵ نتایج مدل رگرسیون خطی

مدل رگرسیون خطی به عنوان ساده‌ترین مدل برای شروع کار انتخاب شد. پس از آموزش مدل روی ۸۰ درصد داده‌ها، نتایج روی بخش تست به شرح زیر است:

- **خطای میانگین مربعات (MSE):** 7.88×10^{16}
- **ریشه خطای میانگین مربعات (RMSE):** حدود ۲۸۰،۷۵۱،۲۵۹ تومان

این میزان خطا نشان می‌دهد که مدل خطی برای پیش‌بینی دقیق قیمت در بازار پرنوسان خودرو کافی نیست و لزوم استفاده از مدل‌های پیچیده‌تر مانند شبکه‌های عصبی در بخش‌های بعدی بررسی خواهد شد.

۶ بخش دوم: پیاده‌سازی شبکه عصبی با کتابخانه Numpy

در این بخش، مطابق با محدودیت‌های ذکر شده در صورت تمرین، یک شبکه عصبی پرسپترون چندلایه (MLP) از پایه و تنها با استفاده از عملیات ماتریسی در کتابخانه Numpy طراحی و پیاده‌سازی شد. در این مرحله استفاده از کتابخانه‌های سطح بالا مانند Keras مجاز نبود.

۱.۶ ساختار و ریاضیات شبکه

برای جلوگیری از مشکل اشباع یا انفجار گرادیان در توابع فعال‌ساز (به دلیل بزرگ بودن ارقام ارقام مربوط به قیمت و کارکرد خودرو)، در ابتدا تمامی داده‌ها با استفاده از روش Min-Max Scaling به بازه صفر تا یک نرمال شدند. ساختار شبکه شامل مراحل زیر است:

- **انتشار رو به جلو (Forward Pass):** ضرب ماتریسی ویژگی‌ها در وزن‌های هر لایه و عبور آن‌ها از تابع فعال‌ساز غیرخطی سیگموئید (Sigmoid).
- **محاسبه خطا:** محاسبه اختلاف میان قیمت پیش‌بینی شده و قیمت واقعی در فضای نرمال‌شده.
- **پس‌انتشار خطا (Backpropagation):** محاسبه مشتقات جزئی خطا نسبت به وزن‌های هر لایه (با استفاده از قاعده زنجیره‌ای مشتق) و به‌روزرسانی تدریجی وزن‌ها با نرخ یادگیری (Learn-Rate) برابر با 0.05.

۲.۶ مقایسه معماری دو لایه و سه لایه مخفی

همان‌طور که در صورت سوال خواسته شده بود، این شبکه یک‌بار با دو لایه مخفی و بار دیگر با سه لایه مخفی آموزش داده شد تا تأثیر عمق شبکه بر دقت پیش‌بینی بررسی شود. نتایج روی داده‌های تست، پس از ۱۵,۰۰۰ دوره آموزش (Epoch) و بازگرداندن مقیاس قیمت‌ها به تومان واقعی (معکوس نرمال‌سازی) به شرح زیر است:

معماری شبکه	خطای MSE	ریشه خطای RMSE (تومان)
شبکه با ۲ لایه مخفی	1.24×10^{17}	۳۵۳,۴۲۳,۹۲۷
شبکه با ۳ لایه مخفی	6.89×10^{16}	۲۶۲,۶۱۷,۹۱۲

جدول ۱: مقایسه عملکرد شبکه‌های عصبی پیاده‌سازی شده با Numpy

تحلیل نتایج: همان‌طور که خروجی‌ها نشان می‌دهند، با افزایش عمق شبکه از دو لایه به سه لایه مخفی، خطای پیش‌بینی به میزان قابل توجهی (حدود ۹۰ میلیون تومان) کاهش یافته است (حتی نسبت به مدل رگرسیون خطی در بخش قبل). این موضوع اثبات می‌کند که توابع حاکم بر قیمت‌گذاری خودرو دارای پیچیدگی‌ها و الگوهای غیرخطی متعددی هستند که استخراج آن‌ها نیازمند شبکه‌ای با ظرفیت و عمق بالاتر است.

۷ بخش سوم: پیاده‌سازی شبکه عصبی با Keras و TensorFlow

در این مرحله، به منظور توسعه یک مدل مقیاس‌پذیر و استفاده از ابزارهای مدرن یادگیری عمیق، معماری شبکه عصبی با استفاده از کتابخانه Keras پیاده‌سازی شد. استفاده از این فریم‌ورک امکان بهره‌گیری از بهینه‌سازهای پیشرفته و توابع فعال‌ساز نوین را فراهم می‌کند.

۱.۷ معماری و پارامترهای مدل

• **نرمال سازی پیشرفته:** برخلاف روش دستی، در این بخش از کلاس MinMaxScaler از کتابخانه scikit-learn استفاده شد تا تمام ویژگی‌ها به صورت یکپارچه و امن به بازه $[0, 1]$ نگاشت شوند.

• **ساختار شبکه‌ی متوالی (Sequential):**

۱. **لایه ورودی و پنهان اول:** یک لایه Dense شامل ۱۶ نود. به جای تابع کلاسیک سیگموئید، از تابع فعال‌ساز ReLU استفاده شد تا سرعت همگرایی افزایش یافته و مشکل محو شدگی گرادیان برطرف گردد.

۲. **لایه پنهان دوم:** شامل ۸ نود با تابع فعال‌ساز ReLU.

۳. **لایه خروجی:** یک تک‌نود با تابع فعال‌ساز خطی (Linear) که برای مسائل رگرسیون (پیش‌بینی مقادیر پیوسته مانند قیمت) مناسب‌ترین گزینه است.

• **الگوریتم بهینه‌ساز:** شبکه با استفاده از بهینه‌ساز پیشرفته‌ی Adam کامپایل شد که نرخ یادگیری تطبیقی (Adaptive Learning Rate) را برای هر پارامتر لحاظ می‌کند و عملکرد بسیار سریع‌تر و بهتری نسبت به گرادیان کاهشی استاندارد دارد.

۲.۷ ارزیابی، چالش تنظیم Epoch و پدیده بیش‌برازش

در اجرای اولیه، مدل طی ۱۵۰ دوره (Epoch) با اندازه دسته (Batch Size) برابر با ۴ آموزش دید. خطای محاسبه شده بر روی داده‌های تست برای این حالت به شرح زیر بود:

• **معیار خطای میانگین مربعات (MSE):** 9.16×10^{16}

• **معیار خطای مجذور میانگین مربعات (RMSE):** حدوداً ۳۰۲،۷۲۶،۳۵۵ تومان

با توجه به اینکه خطای این مدل از مدل دست‌ی Numpy (که ۱۵،۰۰۰ دوره آموزش دیده بود و خطای ۲۶۲ میلیونی داشت) بیشتر بود، در یک آزمایش جدید تصمیم گرفته شد تا تعداد دوره‌های آموزش (Epoch) در کراس به ۸۰۰ افزایش یابد تا شبکه فرصت بیشتری برای یادگیری داشته باشد. **بروز بیش‌برازش (Overfitting):** پس از اجرای مدل با ۸۰۰ دوره، مشاهده شد که خطای آموزش (Train Loss) به شدت کاهش یافت (به حدود ۰.۰۱۲ رسید)، اما خطای تست به جای کاهش، **افزایش** یافت و RMSE به رقم ۳۴۰ میلیون تومان رسید!

تحلیل این رفتار: دلیل این اتفاق ترکیب «استفاده از بهینه‌ساز سریع Adam» و «کوچک بودن مجموعه داده (حدود ۱۴۰ رکورد)» است. در مدل Numpy به دلیل استفاده از مشتق‌گیری ساده و نرخ یادگیری ثابت، شبکه بسیار کند یاد می‌گرفت و به ۱۵،۰۰۰ دوره نیاز داشت. اما در Keras، الگوریتم Adam شبکه را به سرعت بهینه‌سازی می‌کند؛ بنابراین اعمال ۸۰۰ دوره آموزش برای این حجم کم از داده، باعث شد شبکه به جای کشف الگوهای کلی بازار، قیمت تک‌تک ماشین‌های آموزشی را «حفظ کند» (Memorization). در نتیجه توانایی تعمیم خود را روی داده‌های تست از دست داد. برای غلبه بر این مشکل و بهره‌گیری از حداکثر ظرفیت شبکه‌های عمیق بدون افتادن در دام بیش‌برازش، در بخش نهایی از تکنیک‌های منظم‌سازی (Regularization) در یک معماری DNN استفاده خواهیم کرد.

۸ بخش چهارم: شبکه عصبی عمیق (DNN) و مقابله با بیش‌برازش

برای حل چالش بیش‌برازش (Overfitting) که در بخش قبل مشاهده شد و به منظور ارتقای عملکرد مدل روی داده‌های محدود، یک شبکه عصبی عمیق (Deep Neural Network) با معماری پیشرفته‌تر پیاده‌سازی گردید.

۱.۸ طراحی معماری DNN و تکنیک‌های تنظیم (Regularization)

معماری این شبکه شامل سه لایه مخفی به ترتیب با ۳۲، ۱۶ و ۸ نود است. برای جلوگیری از حفظ کردن داده‌ها، دو تکنیک اساسی در معماری شبکه لحاظ شد:

۱. **لایه دراپ‌اوت (Dropout):** پس از لایه‌های مخفی اول و دوم، یک لایه Dropout با نرخ 0.2 اضافه شد. این لایه در هر مرحله از آموزش، به صورت تصادفی ۲۰ درصد از نرون‌ها را غیرفعال می‌کند تا شبکه به مسیرهای خاص وابسته نشده و قدرت تعمیم‌پذیری (Generalization) آن افزایش یابد.

۲. **توقف زودهنگام (Early Stopping):** به جای تعیین یک عدد ثابت و بزرگ برای Epoch که منجر به بیش‌برازش می‌شود، از یک Callback استفاده شد. با اختصاص ۲۰ درصد از داده‌های آموزش به عنوان داده‌های ارزیابی (Validation)، شبکه به صورت زنده رفتار خود را مانیتور می‌کند.

۲.۸ نتایج نهایی مدل عمیق

با اجرای مدل با پیکربندی جدید، مکانیزم Early Stopping متوجه شد که پس از حدود ۹۴ دوره (Epoch)، مدل به بهترین نقطه همگرایی خود رسیده است. بنابراین آموزش را به صورت خودکار متوقف کرد و وزن‌های بهترین دوره را بازگرداند. ارزیابی این شبکه بر روی داده‌های تست نهایی به شرح زیر است:

• معیار خطای میانگین مربعات (MSE): 6.82×10^{16}

• ریشه خطای میانگین مربعات (RMSE): حدوداً ۲۶۱،۳۳۹،۴۳۳ تومان

۹ جمع‌بندی و مقایسه نهایی مدل‌ها

برای ارائه یک دید کلی از عملکرد روش‌های مختلف بررسی شده در این پروژه، نتایج ارزیابی تمامی مدل‌ها بر روی داده‌های تست در جدول زیر خلاصه و مقایسه شده است:

نوع معماری و مدل	خطای MSE	RMSE (تومان)
رگرسیون خطی (مدل پایه)	7.88×10^{16}	۲۸۰،۷۵۱،۲۵۹
شبکه عصبی با Numpy (۲ لایه مخفی)	1.24×10^{17}	۳۵۳،۴۲۳،۹۲۷
شبکه عصبی با Numpy (۳ لایه مخفی)	6.89×10^{16}	۲۶۲،۶۱۷،۹۱۲
شبکه عصبی Keras (تنظیمات اولیه - ۱۵۰ دوره)	9.16×10^{16}	۳۰۲،۷۲۶،۳۵۵
شبکه عصبی عمیق Keras DNN (Dropout + Early Stop)	6.82×10^{16}	۲۶۱،۳۳۹،۴۳۳

جدول ۲: مقایسه نهایی عملکرد مدل‌های یادگیری ماشین و یادگیری عمیق در پیش‌بینی قیمت خودرو

همان‌طور که در جدول بالا (و پررنگ شدن کمترین خطا) مشخص است، بهترین عملکرد متعلق به شبکه عصبی عمیق (DNN) مجهز به تکنیک‌های مقابله با بیش‌برازش است. این مقایسه به وضوح نشان می‌دهد که استفاده از شبکه‌های عمیق به تنهایی ضامن موفقیت نیست، بلکه ترکیب معماری عمیق با تکنیک‌های منظم‌سازی (Regularization) کلید دستیابی به بالاترین دقت در مسائل رگرسیون پیچیده دنیای واقعی است.